

Progressive Learning Agent in a Simulated Environment

Final report

Submitted for the BSc (Hons) Computer Science

600091 Honours Stage Project 2025/26

12th May 2026

by

Muhammad Aysh Khan

Student Number: 202408867

Word count: 14206

Abstract

This project investigates whether layering multiple instinct-driven reward structures leads to improved learning performance and more realistic behavioural patterns in a reinforcement learning agent, compared to training a single agent with a unified objective function. The motivation is drawn from biological systems, in which survival-relevant drives such as hunger, curiosity, and self-preservation coexist and compete simultaneously, producing flexible adaptive behaviour without any single overarching objective.

Three instinct modules were designed and implemented as independent agents operating within custom OpenAI Gymnasium environments: a hunger instinct directing the agent toward food resources while avoiding danger, an exploration instinct rewarding systematic coverage of unseen grid cells, and a survival instinct managing energy levels and proximity to safe zones. Each module was trained independently using tabular Q-learning with exponential epsilon decay and a relative state encoding scheme that reduced state spaces by more than an order of magnitude compared to absolute coordinate encoding.

A multi-instinct agent was then constructed by combining all three reward signals through weighted linear scalarisation. This agent was trained across four configurations: exploration only, hunger only, survival only, and all three combined with equal weights, each for 2,000 episodes in a shared 10x10 grid environment. Experimental results show that individual specialist agents outperform the combined agent on their specific sub-task metrics. The hunger specialist achieved a 77% food-collection success rate in its dedicated environment, compared to an average of less than half a food item per episode for the hunger-only multi-instinct configuration. The exploration specialist achieved 36 to 49% grid coverage in testing, well above the combined agent.

However, the all-combined configuration was the only configuration that ever achieved the survival success condition, defined as maintaining positive energy for at least 30 consecutive steps while reaching the safe zone at least once. Individual instinct configurations failed to meet this composite criterion even when survival was the sole instinct active. This demonstrates a form of emergent composite behaviour in which the interaction of multiple reward signals produces outcomes not achievable by any single signal in isolation.

These findings suggest that instinct layering produces qualitatively richer behaviour at the cost of lower peak performance on any individual sub-task, reflecting a genuine trade-off between specialisation depth and behavioural breadth. The result depends on which success criterion is applied: for single-metric optimisation,

specialisation wins; for composite survival requiring coordination across multiple drives simultaneously, combination is necessary.

The project is implemented entirely in Python using OpenAI Gymnasium, tabular Q-learning, and NumPy, within a single reproducible Jupyter notebook. The architecture is modular, permitting new instinct layers to be added without modifying the core environment or agent code. The state representation employs relative directional encoding, reducing the hunger agent state space from 4,474 keys under absolute encoding to 392 keys under relative encoding, enabling convergence within the available training budget.

1 Introduction

1.1 Background to the Project

The dominant paradigm in applied reinforcement learning research trains a single agent to maximise a single reward function. This approach has produced remarkable results in domains ranging from board games to protein structure prediction, but it relies on a reward function that already encodes everything the designer considers important. Designing such a function correctly is non-trivial: the reward must be dense enough to provide a learning signal in early training, consistent enough not to produce unintended optima, and stable enough not to change the meaning of past experience as the agent learns.

Biological agents do not operate this way. A foraging animal is simultaneously hungry, curious about its environment, and alert to predators. These drives do not resolve into a single unified objective; they compete and interact at every moment, and the animal's behaviour is the result of that interaction. When hunger is strong and threat is low, the animal forages. When threat is high, it retreats to safety regardless of hunger. When satiated and safe, it explores. This architecture is not designed from the top down; it emerges from the independent operation of multiple drive systems that have been shaped by evolution to be relevant to survival.

Computational approaches to reinforcement learning have explored intrinsic motivation as an add-on to extrinsic reward signals, and multi-agent systems in which multiple independent agents share an environment. What has received comparatively little attention is an architecture in which multiple heterogeneous drives are modelled as separate reward-generating modules that operate in parallel within a single agent body, each pursuing its own objective without a higher-level objective to mediate between them. This project constructs such an architecture and evaluates it empirically.

Three separate instinct modules are constructed, each with its own reward function and its own policy. These modules are first trained and evaluated independently in purpose-built environments, establishing individual performance baselines. They are then combined within a shared multi-instinct environment using weighted linear scalarisation, and the combined agent is evaluated alongside single-instinct configurations of the same multi-instinct agent architecture. The central question is whether combining instincts produces behaviours that are qualitatively different from, and in some respects superior to, the behaviours produced by any individual instinct.

The simulation platform used is OpenAI Gymnasium (Brockman et al., 2016). An initial attempt to use Unity ML-Agents as the simulation environment was abandoned after the toolchain proved incompatible with the target hardware, and that decision is discussed in Section 3.3. Gymnasium was selected as the replacement because it provides a clean Python-native interface that integrates directly with the scientific Python stack and imposes no external build dependencies. Custom environments inherit from the `gym.Env` base class and implement a standard reset and step interface, ensuring compatibility with any algorithm that conforms to the Gymnasium API.

1.2 Aims and Objectives

The primary aim of the project is to investigate whether layering basic instinct-driven reward structures leads to improved learning performance and more realistic behavioural patterns in a reinforcement learning agent, compared to training with a single unified objective.

The key objectives underpinning that aim are as follows:

- Design a modular 2D grid world environment that supports multiple simultaneous instincts within a single shared simulation space.
- Implement three independent instinct layers covering hunger-directed resource seeking, novelty-driven exploration, and danger-avoiding survival behaviour.
- Train agents under individual and combined instinct conditions and record quantitative performance metrics throughout.
- Evaluate learning outcomes using reward convergence, learning speed as measured by epsilon decay progress, and task-specific success rates.
- Compare the performance of single instinct specialist agents against the combined multi-instinct agent, both within their own environments and within the shared environment.
- Identify and discuss any emergent behaviours in the combined agent that were absent from individual instinct configurations.

1.3 Research Question

The research question guiding this project is; does layering basic instinct-driven reward structures lead to improved learning performance and more realistic behavioural patterns in a reinforcement learning agent, compared to training with a single unified objective?

This question is intentionally broad because the answer is likely to be nuanced. Improved performance on a single metric and more realistic behavioural patterns are not the same thing, and the project is designed to distinguish between them. An

agent that collects food efficiently but ignores all danger and never explores is not exhibiting realistic behaviour; it is exhibiting optimal behaviour on a narrow metric. The evaluation therefore considers both quantitative metrics and qualitative observations about the character of the learned policy.

2 Literature Review

2.1 Foundations of Reinforcement Learning

Reinforcement learning is a branch of machine learning in which an agent learns to select actions by interacting with an environment and receiving feedback in the form of reward signals (Sutton and Barto, 2018). Unlike supervised learning, no labelled examples are provided; the agent must discover which actions lead to good long-term outcomes through trial and error. The foundational distinction between reinforcement learning and other learning paradigms is that the feedback signal is evaluative rather than instructive: the agent learns how good an action was, not what the correct action should have been.

The mathematical framework underlying most reinforcement learning is the Markov Decision Process. A Markov Decision Process consists of a set of states, a set of actions, a transition function that determines the probability of reaching each state after taking an action, and a reward function that returns a scalar value for each transition. The agent seeks to find a policy, a mapping from states to actions, that maximises the expected cumulative discounted reward over time. The discount factor γ controls how much the agent values immediate rewards relative to future ones; a value close to 1 places high weight on long-term consequences, while a value close to 0 makes the agent short-sighted.

The value function associated with a policy assigns to each state the expected cumulative reward the agent will receive if it follows that policy from that state onward. Similarly, the action-value function assigns to each state-action pair the expected return when taking that action in that state and then following the policy. These two quantities are related by the Bellman equations, recursive consistency conditions that any optimal value function must satisfy. Dynamic programming methods solve the Bellman equations directly when the transition function is known, producing the optimal policy without learning. When the transition function is unknown, as in most practical settings, the agent must estimate value functions from experience.

Q-learning, proposed by Watkins and Dayan (1992), is a specific temporal difference method that learns the value of taking each action in each state without

needing to follow the policy being evaluated. It maintains a table of Q-values indexed by state and action pairs and updates them according to the temporal difference error after each transition. The key property of Q-learning is that it converges to the optimal Q-function regardless of the behaviour policy used to collect experience, provided that every state-action pair is visited infinitely often in the limit. This off-policy property makes it suitable for epsilon-greedy training, where the exploration policy differs from the greedy policy being learned.

A persistent challenge in reinforcement learning is the exploration versus exploitation trade-off. The agent must balance trying unfamiliar actions, which might lead to better long-term outcomes, against exploiting the actions it already knows to be good. Epsilon greedy selection addresses this by choosing a random action with probability epsilon and the greedy action otherwise. Over training, epsilon is decayed, gradually shifting the balance toward exploitation. The rate of decay must be tuned carefully: decay too fast and the agent commits to a suboptimal policy before exploring the state space adequately; decay too slowly and the agent wastes training episodes on random behaviour after a good policy has already been learned.

An important limitation of tabular Q-learning is the requirement for the full state space to be visited during training. When states are represented as raw grid coordinates, the state space grows quickly: an 8x8 grid with three objects produces on the order of several thousand unique states, and training is unlikely to visit each state enough times for Q-values to converge reliably. Dimensionality motivated both the deep reinforcement learning methods and the state abstraction approach used. State abstraction groups multiple raw states into a single abstract state when they are functionally equivalent in terms of the optimal action, reducing the effective state space size (Li, Walsh and Littman, 2006).

2.2 Deep Reinforcement Learning

The limitation of tabular Q-learning to small, discrete state spaces was addressed by the development of deep reinforcement learning, in which a neural network is used to approximate the Q-function rather than storing it in an explicit table. The Deep Q-Network algorithm, introduced by Mnih et al. (2015), combined Q-learning with a convolutional neural network and two stabilising mechanisms: experience replay, which stores past transitions and samples mini-batches of them during training to reduce correlation between consecutive updates, and a separate target network, which provides stable Q-value targets for a fixed number of steps before being synchronised with the online network. Together these mechanisms allowed a

single agent to learn to play forty-nine Atari games from raw pixel input at human level or better.

Deep reinforcement learning has since expanded into policy gradient methods, actor-critic architectures, and model-based approaches, each addressing different aspects of the sample efficiency and stability challenges that arise when using neural networks as function approximators. Proximal Policy Optimisation, developed by Schulman et al. (2017), introduced a clipped surrogate objective that prevents overly large policy updates during training, producing a reliable and well-performing algorithm suitable for both continuous and discrete action spaces. Actor-critic methods maintain separate networks for the policy and the value function, allowing the value network to provide a lower-variance baseline for the policy gradient estimate. Chadi and Mousannif (2023) provide an accessible survey of the progression from basic Q-learning to proximal policy optimisation, tracing the specific architectural decisions that made each successive algorithm more stable or more sample-efficient than its predecessor.

Ghasemi et al. (2025) offer a broader survey of modern reinforcement learning algorithms and their practical limitations, noting that deep reinforcement learning methods typically require orders of magnitude more environment interactions than tabular methods per unit of policy quality, because gradient-based optimisation of neural networks is inherently noisy and requires many steps to average out that noise. This observation is relevant to the current project, which operates under a strict computational budget. Tabular Q-learning was deliberately chosen over deep methods precisely because each tabular update is exact: there is no function approximation error, and convergence can be verified directly by inspecting Q-value differentiation across actions in sample states.

The choice of tabular Q-learning is therefore not a retreat from the state of the art but a deliberate match between method and constraints. The research question in this project is about the architectural effect of combining multiple instinct reward signals, not about achieving the highest possible absolute performance. Tabular methods produce interpretable Q-tables that can be inspected directly, making it possible to confirm that the agent has learned a qualitatively different policy under combined instincts compared to individual ones. A neural network policy does not afford this transparency without additional explainability tooling.

2.3 Intrinsic Motivation and Curiosity-Driven Exploration

Reinforcement learning agents often fail to learn anything useful when external rewards are sparse or delayed. In such settings, the agent may not encounter a

positive reward signal early in training at all, leaving Q-values uniformly zero and providing no gradient for the policy to follow. Intrinsic motivation addresses this by supplementing the extrinsic reward signal with an internally generated reward that depends on the agent's own state or the properties of the environment, rather than on any externally defined task completion criterion.

The concept of intrinsic motivation originates in psychology, where it refers to engagement with an activity for its own sake rather than for an external incentive. Drive-reduction theory, associated with Hull (1943) and later extended by Berlyne (1960), proposes that organisms experience drives such as hunger, thirst, and curiosity as aversive tension states that motivate behaviour aimed at reducing them. From this perspective, exploration is not merely a means to finding reward; it is itself a drive that produces a form of reward when novelty is encountered. This biological framing motivates the exploration instinct in the current project: the agent receives reward for visiting new cells not because the researcher has decided that coverage is valuable, but because coverage is modelled as an intrinsically motivated behaviour analogous to curiosity.

In computational reinforcement learning, intrinsic motivation most commonly takes the form of novelty-based rewards or prediction-error-based rewards. Oudeyer and Kaplan (2009) provide a taxonomy of intrinsic motivation approaches and argue that learning progress, the rate at which the agent reduces its prediction error about the environment, is a more informative signal than raw novelty because it is zero in regions that are either already well understood or completely incomprehensible. Their framework connects computational approaches to developmental psychology and ethology, grounding the technical definitions in a broader theory of how biological organisms allocate attention and effort.

Pathak et al. (2017) implemented a curiosity-driven exploration system in which an agent receives an intrinsic reward proportional to its prediction error about the consequences of its own actions. A self-supervised forward model is trained alongside the policy to predict the next state given the current state and action. The prediction error of this model serves as the intrinsic reward: large errors indicate that the agent is in an unfamiliar or complex part of the environment, and the agent is therefore incentivised to spend time there. This approach generalises to continuous state spaces and produces structured exploration behaviour in environments such as Super Mario Bros. where external rewards are extremely sparse.

More recent work has examined how large pre-trained models can serve as sources of intrinsic motivation. Andres and Del Ser (2024) found that foundation models can

generate meaningful novelty signals even in environments where hand-designed intrinsic rewards are difficult to specify. This is a promising direction for transferring knowledge from large pre-training datasets into reinforcement learning settings without requiring reward engineering, but it introduces its own complexity in terms of computational cost and the risk that the foundation model's notion of novelty does not align with what is useful for the specific task.

The exploration instinct in this project implements a simpler but well-established form of novelty-based intrinsic motivation. The agent receives a fixed positive reward the first time it visits any grid cell and a penalty for remaining in already visited regions. This directly incentivises coverage without any prediction model, at the cost of not generalising across states. The approach is appropriate for the small grid environment used here, where the total number of cells is small enough that all cells can be visited within a reasonable number of episodes. In a larger environment, a density model or prediction-error approach would be necessary.

2.4 Reward Shaping

Reward shaping is the practice of augmenting the environment reward with additional signals designed to guide the agent toward desired behaviours more quickly than the sparse environment reward alone would allow. Ng, Harada and Russell (1999) established an important theoretical result: any reward shaping function of the form $F(s, s') = \gamma * \phi(s') - \phi(s)$, where ϕ is an arbitrary potential function over states, leaves the optimal policy invariant. That is, adding such a potential-based shaping term to the reward function changes the scale and timing of rewards but does not change which policy is optimal. This result is significant because it provides a principled guarantee that shaping will not distort learning outcomes when the shaping function satisfies the potential-based form.

In practice, many reward shaping implementations do not satisfy the potential-based constraint and may change the optimal policy in unintended ways. The proximity-based shaping used in the hunger instinct in this project, rewarding the agent in proportion to its closeness to the nearest food item, is not strictly potential-based because the nearest food item changes identity as food is collected. This means that the shaping signal can produce discontinuities at food collection events: the agent receives a large positive reward for collecting food, and then the shaping signal resets to reflect the new nearest food item, which may be further away, producing an apparent penalty immediately after collection. This issue was identified during testing and addressed by ensuring that the shaping signal is computed before the environment step, not after it, as described in Section 5.2.1.

A further consideration in reward shaping is the magnitude of the shaping signal relative to the primary task reward. If shaping signals are too small, they provide negligible gradient information in regions far from the goal. If they are too large, the agent may learn to maximise shaping rewards while ignoring the primary task, a phenomenon sometimes called reward hacking. The magnitudes used in this project were chosen by inspection of early training curves: the shaping component was increased until the training reward curve showed a clear upward trend in the first 100 episodes, then held constant. This is an informal calibration procedure; a more rigorous approach would use sensitivity analysis across a range of shaping magnitudes.

2.5 Multi-Agent Reinforcement Learning

Multi-agent reinforcement learning extends the single-agent framework to settings where multiple agents learn and act within the same environment simultaneously (Lowe et al., 2017). The introduction of multiple learning agents creates challenges that do not arise in single-agent settings. The environment is no longer stationary from the perspective of any individual agent, because the actions of other agents change the dynamics that each agent must learn to model. This non-stationarity violates the Markov property that Q-learning convergence proofs depend on, and makes it difficult to guarantee that any individual agent's learned policy will remain optimal as the other agents continue to update their policies.

The multi-agent literature broadly distinguishes between cooperative settings, where agents share a common objective or communicate to coordinate their actions, and competitive settings, where agents have opposing objectives and must find strategies robust to adversarial behaviour. Mixed settings exist in which some agents cooperate and some compete. Each setting produces different learning dynamics and requires different algorithmic approaches. In cooperative settings, the challenge is producing coordinated emergent behaviour from independently learning agents; in competitive settings, the challenge is finding stable equilibrium in which no agent has an incentive to deviate from its current strategy.

Centralised training with decentralised execution is a widely used paradigm for cooperative multi-agent settings. During training, a centralised critic has access to the joint observations and actions of all agents and provides a more informative learning signal than any individual agent could compute from its own observations alone. During execution, each agent acts independently using only its own local observation. Lowe et al. (2017) introduced the Multi-Agent Deep Deterministic Policy Gradient algorithm, which implements centralised training with decentralised execution for continuous action spaces and demonstrated

coordinated behaviours in cooperative-competitive tasks involving both team and individual objectives.

The architecture in this project is not strictly a multi-agent system in the MARL sense, because there is only one agent body and one Q-table in the combined configuration. The parallel is structural rather than literal: multiple reward-generating modules operate simultaneously within a single agent, much as multiple agents in MARL produce simultaneous action signals within a single environment. The distinction is important for interpreting results: the competition between instinct signals in this project is resolved algebraically at each step by the scalarisation function, not by negotiation or communication between independent learning systems.

2.6 The Gap: Heterogeneous Instinct Agents in Parallel

Despite extensive research into both intrinsic motivation and multi-agent systems, a clear gap remains in the literature. There is limited work exploring architectures in which multiple heterogeneous agents each pursue a fixed, non-negotiable intrinsic objective while operating in parallel within the same environment, without any higher-level objective mediating between them. This is distinct from both standard MARL, where agents share or oppose a task-level objective, and from intrinsic motivation research, where a single agent uses an intrinsic bonus to accelerate learning of a primary extrinsic task.

A useful analogy is that of a biological organism. Different drives such as hunger, curiosity, and fear of predators exist simultaneously within the same animal and frequently compete for priority. The organism does not resolve these drives into a single utility function and then optimise it; instead, it responds to whichever drive is most salient given the current context, with salience determined by both the strength of the drive and the current environmental state. Hunger overrides curiosity when food is scarce and the animal is depleted; curiosity overrides safety when the animal is satiated and no predators are nearby. This context-sensitive competition produces qualitatively different behaviour than any single-drive optimisation would produce.

Standard MARL research focuses on agents coordinating toward a joint objective or competing against opponents, both of which assume some defined relationship between the agents' goals (Lowe et al., 2017). Intrinsic motivation research typically enhances the exploration or learning efficiency of a single-objective agent rather than modelling drives as permanent, equal-priority objectives in their own right (Pathak et al., 2017). Neither framework captures the architecture described above,

in which multiple drives are genuinely co-equal and the resulting behaviour is determined by their interaction rather than by any external objective function.

There is related work in the field of option-based hierarchical reinforcement learning, where a high-level policy selects between temporally extended sub-policies called options (Sutton, Precup and Singh, 1999). This architecture shares some structural similarities with the instinct framework, in that multiple policies can be active at different times and a higher-level mechanism determines which one controls the agent's behaviour. However, in the options framework the high-level policy is itself a learned component trained to maximise the same objective as the sub-policies, and the sub-policies are selected sequentially by the meta-policy rather than operating simultaneously. The instinct framework considered here has no learned arbitrator; all three instincts contribute simultaneously to the reward at every step, and none takes priority over the others at any point.

2.7 Positioning of This Work

This project directly addresses the gap identified above by proposing and implementing a modular, instinct-driven reinforcement learning framework in which three biological drives, hunger, exploration, and survival, are modelled as independent Q-learning agents. Each instinct maintains its own policy, its own reward function, and its own state encoding optimised for the objects relevant to that drive. The three instincts are then combined through linear scalarisation in a shared environment, and the combined agent is evaluated empirically against individual instinct baselines.

The tabular Q-learning approach is deliberately chosen over deep reinforcement learning methods to keep the internal workings of each policy transparent and inspectable. This transparency is important for the research question: to confirm that the combined agent exhibits genuinely different behaviour from any individual configuration, it is necessary to inspect not just performance metrics but the character of the learned policy itself. Deep networks encode policies implicitly in millions of parameters; tabular Q-tables encode policies explicitly in a dictionary that can be printed, sorted, and compared entry by entry.

3 Requirements

3.1 Product Requirements

The system must provide a set of simulated grid world environments and reinforcement learning agents that allow instinct-driven behaviours to be trained, evaluated, and compared in a reproducible manner. The primary user is the

researcher, who requires quantitative training metrics, the ability to inspect individual agent policies, and evidence that experimental results are stable across full notebook re-executions.

The system is not a user-facing product. There is no graphical user interface beyond the training plots generated by matplotlib and the animated visualisation produced for demonstration purposes. The primary deliverable is the Jupyter notebook itself, which functions simultaneously as the implementation, the experimental record, and the documentation of results. The notebook must be self-contained: running it from start to finish with no prior state should reproduce all training curves, Q-table statistics, and test results reported in this document.

Modularity is a core product requirement. The three instinct environments and their associated agents must be independently executable, so that a failure or design change in one instinct does not require modifying the others. This requirement influenced the decision to define each environment and agent class within its own notebook cell, with no shared module-level state between cells. Each cell imports only the standard library packages it needs and defines all classes locally.

3.2 Functional Requirements

3.2.1 Instinct Environments

Three independent custom Gymnasium environments must be implemented, each inheriting from `gym.Env` and implementing the standard reset and step interface. The hunger environment uses an 8x8 grid with three food items and two danger objects spread across the space, with the agent rewarded for collecting food and penalised for contacting danger. The exploration environment uses a 10x10 grid with five obstacles and three landmarks, rewarding the agent for visiting previously unvisited cells. The survival environment uses an 8x8 grid with four danger zones and two safe zones, rewarding the agent for maintaining energy and reaching safety.

A fourth multi-instinct environment must combine elements from all three into a shared 10x10 grid containing food items, danger zones, safe zones, and a visited cell tracker, providing a single environment in which all three instinct reward signals can be computed simultaneously. This environment must support four distinct weight configurations by accepting a weight vector at initialisation, so that any combination of instinct weights can be tested without modifying the environment code.

3.2.2 Agent Architecture

Each instinct agent must implement tabular Q-learning with epsilon greedy exploration and exponential epsilon decay. The Q-table must be implemented as a Python defaultdict initialised to zero, so that unseen states are handled implicitly without pre-enumeration of the state space. State representations must use relative encoding, encoding the direction and approximate distance to each relevant object rather than absolute coordinates. The agent must expose a train method that runs for a configurable number of episodes, a test method that evaluates the learned policy with epsilon fixed at a low value, and a save method that serialises the Q-table and parameters to a pickle file for later reuse.

3.2.3 Training and Evaluation

Each agent must be trainable for a configurable number of episodes and must log per-episode reward, task-specific success metrics, Q-table size, and exploration rate at regular intervals. A separate evaluation phase must run the trained agent with epsilon fixed at 0.05, record per-episode metrics for a configurable number of test episodes, and print a summary of test performance. The evaluation output must be sufficient to determine whether the qualitative results reported in this document are reproduced in a fresh run.

3.2.4 Performance Targets

The hunger agent must achieve a food collection success rate of at least 60% in testing after 1,000 training episodes, where success is defined as collecting two or more of the three available food items per episode. The survival agent must show a measurable reduction in danger hit rate between the first 50 and last 50 training episodes. The exploration agent must demonstrate grid coverage above the level expected from a random walk of the same duration. The multi-instinct combined configuration must achieve the survival success condition in at least one test episode out of three.

3.2.5 Reproducibility

All training results must be reproducible by executing the notebook from start to finish using the jupyter nbconvert execute command. Stochastic variation between runs is expected and acceptable, but the qualitative results, the hunger agent outperforming the no-shaping baseline, the survival agent reducing danger exposure over training, the combined agent achieving survival success where single-instinct configurations do not, must hold consistently. Pre-trained models are saved to pickle files so that the evaluation cells can be run without re-training if required.

3.3 Design Constraints

Unity ML-Agents was evaluated as the initial simulation platform and was found to be incompatible with the target hardware environment. A supported Unity Editor version was successfully installed and launched, confirming that the base toolchain was operational. However, when the ML-Agents Python package was installed and the connection between the Unity editor and the Python training script was established, a persistent version incompatibility error prevented the training session from initialising. Debugging this error required investigating the Unity editor version, the ML-Agents package version, and the Python environment version simultaneously, and no combination was found to resolve all three constraints within a reasonable time budget.

The decision to pivot to OpenAI Gymnasium was taken before committing to a custom Unity environment, which preserved project time for the core research question. This was a deliberate engineering judgement rather than a failure of implementation. Gymnasium provides a Python-native interface that avoids the cross-process communication overhead of Unity ML-Agents, simplifies debugging to a single Python environment, and integrates directly with standard scientific Python packages without any additional serialisation layer.

The project is constrained to CPU-only computation on macOS with Apple Silicon using Python 3.12 and the Anaconda distribution. This limits the practical episode count for each training run, as longer runs take significant wall-clock time. Tabular Q-learning was chosen specifically because each training step requires only a single dictionary lookup and update operation, making it substantially faster per episode than neural network training on CPU. On the hardware used, 2,000 episodes of the multi-instinct agent complete in approximately four minutes, which is acceptable for iterative experimentation. A DQN implementation on the same hardware would require significantly longer per run and would reduce the number of experimental configurations that could be tested within the project timeline.

4 Design

4.1 Environment Design

All environments in this project are implemented as subclasses of `gym.Env`, the base class provided by OpenAI Gymnasium. This requires implementing a `reset` method that returns an initial observation and a `step` method that accepts an action, advances the environment by one timestep, and returns the next observation, the reward, a done flag, and an info dictionary. The action space in all

environments is a Discrete(4) space corresponding to the four cardinal movements: up, down, left, right. Diagonal movement is not permitted. The observation space is defined as a Box space with shape corresponding to the grid dimensions and semantic channels, though in practice the Q-learning agents do not use the raw observation directly; they construct their own abstract state key from the environment's internal state attributes.

The choice of a 2D grid world as the simulation medium reflects a deliberate design decision to keep the environment simple enough that agent behaviour can be fully understood and inspected. Continuous environments or three-dimensional spaces would introduce additional complexity in both the observation representation and the action space, and would require substantially more training to produce convergent policies. Grid worlds have a long history in reinforcement learning research as tractable test beds for algorithmic ideas, and the simplicity of the grid world is an advantage in this project because the research question is about the effect of instinct combination on policy character, not about achieving performance on a complex task.

Each environment uses a three-dimensional numpy array as the observation. The first two dimensions represent the grid, and the third dimension represents semantic channels, where each channel marks the positions of a different object category. This multi-channel design allows the agent to perceive the full environment state in a single observation without needing to interpret raw colour or texture information. The hunger environment has channels for the agent position, food positions, and danger positions. The survival environment adds channels for safe zones and an energy indicator. The multi-instinct environment combines all channels from all three instinct environments.

Table 1 summarises the configuration of all four environments.

Table 1: Environment configurations used in the project.

Environment	Grid size	Objects	Episode length	Terminal condition
Hunger	8x8	3 food, 2 danger	50 steps	Step limit
Exploration	10x10	5 obstacles, 3 landmarks	100 steps	Step limit
Survival	8x8	4 danger, 2 safe zones	60 steps	Step limit / energy exhaustion / survival

Multi-instinct	10x10	3 food, 4 danger, 2 safe zones	60 steps	Step limit / energy / survival
----------------	-------	--------------------------------	----------	--------------------------------

Agent starting positions are randomised at the beginning of each episode to prevent the agent from memorising a fixed route. Food, danger, and safe zone positions are also randomised within constraints that prevent objects from spawning on top of one another. This randomisation is essential for the learned policy to generalise: an agent trained with fixed object positions would learn a route rather than a policy, and would fail completely if object positions changed. The randomisation also means that Q-table states corresponding to specific object configurations will be visited many times across different episodes but never in the same spatial configuration twice, which is the mechanism by which relative encoding achieves high visits-per-state coverage.

Episode length is fixed rather than determined by a terminal condition in most configurations. The hunger and exploration environments run for 50 and 100 steps respectively; the survival environment ends either when the episode step limit is reached, the agent's energy is exhausted, or the agent contacts a safe zone and has survived for at least 30 steps. The conditional terminal condition in the survival environment is necessary to define the survival success criterion: the agent must demonstrate active management of its trajectory toward safety, not merely avoid danger by chance for a fixed number of steps.

4.2 State Representation: From Absolute to Relative Encoding

The state representation used by the Q-learning agent to index into the Q-table is the most consequential design decision in this project. The initial implementation used absolute encoding, in which the state key was constructed from the agent's exact grid row and column, combined with the exact grid positions of all relevant objects. This produced a state key that uniquely identified every distinct configuration of the agent and objects in the grid, which is mathematically correct but practically intractable for the episode counts available.

This produced state spaces of 4,000 to 5,000 unique keys for the hunger agent after 1,000 training episodes. With 50 steps per episode, the total number of transitions was 50,000, giving an average of roughly 10 to 12 visits per unique state. That is insufficient for reliable Q-value convergence. Moreover, many states were visited only once or twice, meaning their Q-values were essentially random and provided no useful information to the greedy policy. The result was an agent that learned a policy in the most frequently visited states but behaved randomly in the rest.

Relative encoding resolves this by storing how each object relates to the agent rather than where each object is in absolute terms. Each object is encoded as a triple consisting of the row direction from agent to object (taking one of the values minus one, zero, or plus one), the column direction (taking the same three values), and a distance bin (near, medium, or far, defined by thresholds in Manhattan distance). This produces nine possible directional-distance combinations per object, and a state key for the hunger agent with two objects of each type produces a much smaller state space.

This encoding collapses an enormous number of physically distinct configurations into a small number of abstract states. The hunger agent settled at 392 unique state keys after 1,000 episodes, compared to 4,474 under absolute encoding. With 50,000 total transitions distributed across 392 states, average visits per state rose to approximately 127. At this level, Q-values have been updated enough times for the learning rate to smooth out early random variation and converge to stable estimates. The qualitative difference in the learned policy between absolute and relative encoding agents is substantial, as documented in the evaluation section.

For the exploration agent, the state key encodes the eight immediately surrounding cells of the agent as a visited-or-unvisited binary string, combined with a five-bin quantisation of the overall exploration progress as a fraction of the total grid cells. This gives the agent a local sense of which neighbours are unexplored and a global sense of how far through the exploration task it is. The eight-cell neighbourhood binary string produces 256 possible patterns, and combined with five progress bins, the theoretical maximum state space is 1,280. In practice, the exploration agent settled at 580 unique keys, reflecting the fact that not all 256 neighbourhood patterns are reachable from all progress levels.

The relative encoding approach is consistent with the state abstraction literature, which distinguishes between states that are functionally equivalent in terms of the optimal action and argues that a good abstraction preserves this equivalence while minimising the number of abstract states (Li, Walsh and Littman, 2006). The directional relative encoding used here is a form of bisimulation-inspired abstraction: two absolute configurations are mapped to the same abstract state if the relevant objects stand in the same directional and distance relationship to the agent. This is a reasonable functional equivalence criterion under the assumption that the optimal action depends on direction and approximate distance to relevant objects, not on absolute position.

4.3 Agent Architecture and Q-Learning Parameters

Each agent in the project implements the same core Q-learning algorithm, differing only in the state key construction method and the reward signal it receives. The update rule applied after each transition is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha * [r + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$$

where s is the current abstract state, a is the action taken, r is the reward received, s' is the next abstract state, α is the learning rate, and γ is the discount factor. The term in square brackets is the temporal difference error, the difference between the current Q-value estimate and the bootstrapped target. This error drives the update: when the target is higher than the current estimate, the Q-value for the taken action is increased; when lower, it is decreased.

The learning rate α is set to 0.1 across all agents. A higher learning rate would cause the agent to overwrite previous Q-value estimates too aggressively, losing information from earlier training; a lower rate would slow convergence unnecessarily. The value 0.1 is a standard choice for tabular Q-learning in small state spaces and was not tuned experimentally in this project, though future work could conduct a sensitivity analysis to confirm it is near-optimal for these environments. The choice assumes that each state-action pair is visited many times during training, so that the learning rate can be low without causing the Q-values to converge too slowly. This assumption is satisfied under relative encoding, where visits-per-state averages above 100.

The initial exploration rate ϵ is set to 1.0, meaning the agent acts entirely randomly at the start of training. ϵ decays by a factor of 0.995 after each episode, reaching the minimum value of 0.05 after approximately 590 episodes. The minimum of 0.05 ensures that the agent continues to occasionally try random actions even after the policy has converged, providing a small amount of ongoing exploration that prevents the policy from becoming trapped in a locally optimal trajectory. The 0.995 decay factor was chosen to allow approximately 590 episodes of progressively less random exploration before the agent shifts to near-greedy behaviour. This is appropriate for episode budgets of 800 to 2,000 episodes, leaving substantial near-greedy episodes for policy consolidation after the exploration phase ends.

4.4 Reward Function Design

The reward function for each instinct is implemented as a separate function that takes the environment state and the outcome of a step as inputs and returns a scalar reward. This separation from the environment logic means that any reward

component can be modified, removed, or replaced without changing the environment dynamics. It also makes the reward function explicit and human-readable, which is important for verifying that the agent is learning from the intended signal.

Table 2 lists the reward components for each instinct. The magnitudes were chosen to ensure that the primary task signal, collecting food, avoiding danger, or surviving, dominates over the shaping signals, while the shaping signals are large enough to provide a meaningful gradient in states where the primary signal is zero.

Table 2: Reward components per instinct.

Instinct	Event	Reward
Hunger	Food collected	+10.0
Hunger	Danger contacted	-8.0
Hunger	Proximity shaping (per step)	0.0 to +0.4
Exploration	New cell visited	+2.0
Exploration	Already visited cell (penalty)	-0.5
Survival	Step survived with energy	+1.0
Survival	Danger hit (energy loss)	-5.0
Survival	Safe zone reached	+10.0

The proximity-based shaping in the hunger instinct deserves particular explanation. At each step, the distance from the agent to the nearest remaining food item is computed in Manhattan distance. This is converted to a value between 0.0 and 0.4 by computing one minus the ratio of the distance to the maximum possible distance on the grid. This shaping value is added to the step reward regardless of whether food was collected on that step. The effect is that the agent receives a slightly higher reward when it is close to food even without collecting it, which provides a directional gradient pointing toward food in states where the primary reward is zero.

The magnitude ceiling of 0.4 for the proximity shaping was chosen to be substantially smaller than the food collection reward of 10.0 and the danger penalty of 8.0. This ensures that the agent always prefers collecting food or avoiding danger over simply moving closer to food, and never prefers the proximity bonus so much that it circles near a food item without collecting it. The asymmetry between the food reward and the danger penalty reflects the asymmetry in the cost structure of the task: missing a food item is recoverable across many episodes, whereas contacting danger produces an immediate and irrevocable negative outcome within the episode.

4.5 Multi-Instinct Combination Mechanism

The multi-instinct agent combines the three reward signals through weighted linear scalarisation. At each step, the total reward is computed as the sum of the hunger reward multiplied by the hunger weight, plus the exploration reward multiplied by the exploration weight, plus the survival reward multiplied by the survival weight. This total reward is then used as the single reward input to the Q-learning update. The agent has no explicit knowledge that the reward is a combination of three signals; from the Q-learning algorithm's perspective, it receives a single scalar reward at each step and updates its Q-values accordingly.

Linear scalarisation was chosen as the combination mechanism because it is the simplest possible approach and produces a clearly interpretable result. In the all-combined configuration, all three weights are set to 1.0, treating each instinct signal as equally important. This is unlikely to be optimal because the three signals have different magnitudes and different frequencies: the survival step reward of 1.0 accumulates every step, while the food collection reward of 10.0 is only received when food is actually collected. The net effect is that the survival signal contributes a background of small positive rewards, the exploration signal provides bonuses for new cells, and the hunger signal contributes large but infrequent positive rewards and large negative rewards when danger is contacted. These different magnitudes mean that equal weights do not produce genuinely equal influence; the food and danger signals dominate individual step decisions because of their large magnitude.

The multi-objective reinforcement learning literature addresses the general problem of optimising agents with multiple reward signals. Roijers et al. (2013) provide a survey of multi-objective sequential decision-making methods, covering both scalarisation approaches and Pareto-based approaches that compute the full set of policies that are not dominated on any objective. The current project uses the simplest scalarisation approach. A Pareto-based approach would produce a set of policies along the Pareto frontier between the three objectives, providing a richer picture of the trade-off between instinct performance levels, but at substantially greater computational cost.

4.6 Experimental Design

The experimental design is structured in two phases. The first phase trains and evaluates each instinct agent independently in its dedicated environment. This establishes individual baselines for the hunger, exploration, and survival instincts and allows the quality of each agent's learned policy to be assessed without the confounding effects of the other instincts or the larger multi-instinct environment.

The independent baselines are important for interpreting the multi-instinct results: if the hunger-only multi-instinct configuration performs poorly on food collection, it is necessary to know whether this reflects the general difficulty of the task or a specific limitation of the combined architecture.

The second phase uses the multi-instinct environment and trains agents under four different weight configurations: exploration only, hunger only, survival only, and all three combined with equal weights. Each configuration is trained for 2,000 episodes. The same environment and the same agent class are used for all four configurations; only the weight vector passed to the environment at initialisation differs. This controls for any differences between the multi-instinct environment and the individual specialist environments, making the comparison between configurations as clean as possible.

For each agent, training metrics including episode reward, task success, Q-table size, and exploration rate are logged at regular intervals. After training, a testing phase runs the agent for three episodes with epsilon fixed at 0.05 and records per-episode performance. Three test episodes is a small sample. The training metrics are plotted using matplotlib and embedded directly in the notebook output, so that the training behaviour is visible alongside the final performance numbers. Key diagnostic statistics, Q-table size, visits per state, and sample Q-value vectors, are printed at the end of training for each agent.

5 Implementation and Testing

5.1 Implementation Overview

The entire project is implemented in a single Jupyter notebook, AdamProject.ipynb, using Python 3.12. The notebook contains seven code cells and one markdown analysis cell. The cells are ordered to match the experimental structure: Cell 0 implements the hunger environment and agent, Cell 1 implements the exploration environment and agent with absolute encoding, Cell 2 implements the exploration agent with relative encoding for comparison, Cells 3 and 4 implement the survival environment and agent in two versions, and Cells 5 and 6 implement the multi-instinct environment and the four-configuration comparison. The markdown cell, inserted between Cells 5 and 6, provides the written analysis of the research question.

Each cell is self-contained. The environment and agent classes, the training function, and the testing and evaluation functions are all defined within a single cell, so any individual instinct can be run and inspected without executing the

preceding cells. This design choice simplifies debugging and demonstration but does introduce some code duplication: the Q-learning update rule, the epsilon decay schedule, and the training loop structure are repeated in each cell. Refactoring these into a shared module was considered but rejected because it would create a cross-cell dependency that reduces self-containment.

5.2 Key Implementation Challenges

5.2.1 Broken Reward Function in the Hunger Agent

The most significant implementation problem encountered during the project was a bug in the original hunger reward function. In the initial version, the reward was computed after calling `env.step()`, which advanced the environment and moved the agent to its new position. The reward function then extracted the agent's current position from the environment state to compute the proximity shaping signal. The result was that the proximity signal was computed from the new position, one step ahead, rather than from the position before the action was taken.

The symptom was counterintuitive. The training curve showed steady reward accumulation across episodes, suggesting that the agent was learning. But testing revealed that the trained agent performed no better than a random walk on food collection. The Q-values had converged to values that reflected the distorted reward signal rather than the intended one, and the policy they implied was not a food-seeking policy. This diagnostic pattern, good training curves but poor testing performance, is a known signature of reward misspecification in reinforcement learning.

The fix was to compute the proximity shaping signal from the current agent position before the step is taken, using only the pre-step state. The shaping value is then added to the environment step reward rather than replacing it. This is computed as one minus the ratio of the Manhattan distance to the nearest food item to the maximum possible Manhattan distance on the grid, clipped to the range zero to 0.4. After this fix, testing performance improved substantially and the Q-values differentiated clearly between actions pointing toward food and actions pointing away from it.

5.2.2 State Space Management

As described in Section 4.2, absolute state encoding produced state spaces of several thousand unique keys that could not be adequately covered within the training episode budget. The switch to relative encoding was implemented separately for each agent, as the relevant objects differ across instincts: the hunger agent encodes food and danger directions, the exploration agent encodes local

neighbourhood visited status, and the survival agent encodes danger and safe zone directions together with energy level.

In each case, the relative encoding was validated by inspecting the Q-table size after training and computing the average number of visits per unique state. The hunger agent settled at 392 states with approximately 127 average visits, the exploration agent at 580 states with approximately 69 average visits, and the survival agent at approximately 7,700 states with approximately 15 average visits. The survival agent's larger state space reflects the greater number of objects it must track simultaneously and the inclusion of energy level as a state variable, which effectively multiplies the state space by the number of energy bins used. Even at 7,700 states, the per-state visit count is sufficient for Q-values to differentiate meaningfully between actions, as verified by inspecting sample Q-value vectors.

5.2.3 Episode Count Calibration

Determining the appropriate number of training episodes for each agent required iterative testing. The initial episode counts, 300 for hunger, 400 for exploration, and 500 for survival, were drawn from preliminary experiments in which the training curves had not yet plateaued. The visits-per-state analysis described above was used to determine when the state space was adequately covered: when average visits per state exceeded approximately 50, Q-values were considered reliable enough to produce a stable policy.

Episode counts were increased to 1,000 for the hunger agent, 800 for the exploration agent, and 2,000 for the survival agent based on this analysis. The multi-instinct V2 agent was also increased from 500 to 2,000 episodes. Each change was verified by re-running the notebook and confirming that the training reward curve had plateaued and that testing performance was qualitatively stable across successive runs. The survival agent required the most episodes because its larger state space required more transitions to achieve adequate coverage, and because the sparse positive reward for safe zone visits means that many episodes early in training contain no positive terminal event.

5.2.4 Version 1 Versus Version 2 Multi-Instinct Agent

Two versions of the multi-instinct agent were implemented and compared. The Version 1 agent used a simpler state representation that did not fully apply relative encoding across all instinct dimensions, resulting in a larger state space and weaker convergence. The Version 2 agent applies the full relative encoding scheme to all objects in the multi-instinct environment, producing a more compact state space and higher visits-per-state coverage. The comparison between V1 and V2 is

documented in the notebook output of Cell 5 and demonstrates that the state representation improvement produced a measurable difference in policy quality, confirming that the encoding design is the primary driver of learning success in these environments.

5.3 Testing Methodology

Each agent is evaluated using a dedicated testing function that runs the agent in the trained environment with epsilon fixed at 0.05. This near-deterministic setting allows the learned policy to be assessed without the noise introduced by random exploration, while the small residual epsilon prevents the agent from being permanently trapped in a locally poor trajectory if a suboptimal Q-value causes it to take a poor action early in the episode. The testing function records the total reward, the task-specific success metric, and the action distribution for each test episode.

The action distribution output is particularly informative for diagnosing policy quality. An agent that has not learned a meaningful policy tends to concentrate actions on a small subset, often favouring a single direction when all Q-values are near zero. A well-trained agent typically shows a more balanced distribution across the four actions, reflecting that the appropriate action genuinely varies across the states encountered during a test episode. Complete concentration on one action in testing is a diagnostic signal that the Q-table has not differentiated between actions and the agent is effectively acting randomly under a fixed deterministic policy.

All training runs were executed by calling `jupyter nbconvert` with the `execute` and `inplace` flags, ensuring that the outputs embedded in the notebook were produced by a full sequential run from a clean kernel state and not selected interactively. This provides a degree of experimental reproducibility, though full reproducibility in the strict sense would require fixing random seeds throughout. The absence of fixed seeds means that re-running the notebook will produce quantitatively different output but qualitatively similar results, as the core findings depend on structural properties of the learning problem rather than on specific numerical outcomes.

6 Evaluation and Discussion of Results

6.1 Baseline Comparison: Absolute Versus Relative Encoding

Cell 1 of the notebook implements a simple exploration agent with absolute state encoding, serving as a baseline. This agent encodes its current grid position as a two-dimensional integer coordinate combined with a flattened representation of the entire visited-cell map. After 300 training episodes, the Q-table contained 5,276

unique state keys with an average of approximately 5.7 visits per state. Grid coverage in testing averaged 22% of the 10x10 grid, which is consistent with a partially directed random walk: better than purely uniform random sampling but well below systematic coverage.

Cell 2 implements the relative-encoding exploration agent trained for 800 episodes. This agent encodes its local neighbourhood visited pattern and global exploration progress, as described in Section 4.2. The Q-table contained 580 unique state keys with approximately 69 average visits. Grid coverage in testing averaged 33%, compared to 22% for the absolute-encoding baseline. The Q-value vectors in the relative-encoding agent showed clear differentiation: states with many unvisited neighbours had high Q-values for the actions pointing toward those neighbours, while states surrounded by visited cells showed preference for the exploration boundary.

This comparison establishes that relative encoding is not simply an implementation detail but has a measurable effect on the quality of the learned policy. The relative-encoding agent achieves higher coverage with fewer unique states and more visits per state, producing a more coherent policy. The 11 percentage point improvement in average test coverage from 22% to 33% represents a meaningful gain in exploration effectiveness, though neither agent approaches the theoretical maximum of systematic grid coverage. The comparison also validates the general principle applied throughout the project: that state abstraction through relative encoding is the key enabler of policy learning within the available episode budget.

6.2 Hunger Instinct Agent

The hunger agent was trained for 1,000 episodes in the 8x8 grid environment with three food items and two danger zones. The training reward curve, embedded in the notebook output of Cell 0, shows a clear upward trend from a baseline of approximately 2.0 per episode in early training to approximately 28.0 in the final 100 episodes. This improvement is driven primarily by increasing food collection rates and decreasing danger contact rates as the proximity shaping guides the agent toward food while the danger penalty discourages contact with hazard zones.

Success rate, defined as collecting two or more of the three food items in a single episode, rose from 40% in the first 100 episodes to 77% in the final 100 episodes. Average food items collected per episode improved from 1.04 in early training to 2.92 in late training. The Q-table stabilised at 392 unique state keys, and the sample Q-value vectors printed at the end of training showed consistent differentiation: states with food nearby and to the right showed higher Q-values for the rightward

action than for any other direction, and states with danger in the immediate forward cell showed negative Q-values for the forward action. These patterns confirm that the agent has learned a meaningful food-seeking, danger-avoiding policy.

Test performance on three episodes showed considerable variability. Episode 2 collected 10 food items over 50 steps, which is possible because food respawns after collection in some test configurations, while Episode 1 collected zero food, with the agent moving Up and Down repeatedly on the same two cells. This extreme variability within three test episodes reflects the sensitivity of the learned policy to starting position: configurations where the agent starts near food produce high collection rates, while configurations where the agent starts far from food rely on the shaping gradient to navigate there, which can fail when the epsilon-greedy noise directs the agent away from the gradient direction in the early steps of the episode.

The remaining failures appear to stem from configurations where the agent starts far from all food items and the proximity signal is not strong enough to overcome the initial random policy. In such cases, the epsilon of 0.05 is not sufficient to rescue the agent from an unfavourable trajectory, and the episode ends with low collection. This is a genuine limitation of the fixed epsilon floor approach: a truly deterministic greedy policy might perform better in these cases, but would be more likely to get trapped in locally optimal but globally suboptimal cycles. The trade-off between exploration floor and policy exploitation is not fully resolved in the current design.

6.3 Exploration Instinct Agent

The exploration agent was trained for 800 episodes in the 10x10 grid environment with five obstacles and three landmarks. Grid coverage improved from 36.4% in the first 50 episodes to 37.1% in the final 50 episodes, a modest improvement that reflects the nature of the exploration task. Early in training, the agent explores randomly and achieves reasonable coverage by chance; late in training, the learned policy produces more systematic coverage but the absolute numbers are not dramatically higher. The key difference between early and late training is the consistency of coverage: late-training episodes show less variance, with the agent reliably covering 30 to 40% of the grid rather than varying between 10% and 60% as in early training.

The Q-table settled at 580 unique state keys. The most common state key pattern in the sample output is a string of zeros for the neighbourhood bitmap combined with an exploration bin of 0, corresponding to the agent being surrounded by unvisited cells early in an episode. Q-values for this state show roughly equal preferences for

all four directions, which is appropriate because all four directions lead to unvisited cells. States with a partially visited neighbourhood show stronger directional preferences, pointing toward the unvisited adjacent cells. This pattern confirms that the agent has learned a local coverage heuristic: prefer unvisited neighbours, with no strong global preference for any direction.

Test performance averaged 33.0% grid coverage across three episodes, with individual episodes ranging from 12% to 49%. The high variance again reflects the small test sample and the stochastic starting position. An episode starting near a corner will allow the agent to spiral outward and achieve high coverage relatively easily; an episode starting at the centre requires the agent to navigate outward in all four directions to cover the grid, which is harder to coordinate within the 100-step episode limit. A larger test sample would allow this positional variability to be averaged out and would produce a more stable estimate of the agent's true average coverage capability.

The epsilon floor for this agent was set at 0.10 rather than 0.05. This means that approximately one in ten actions is still random at the end of training, which makes it difficult to isolate the contribution of the learned policy from residual random exploration. A lower epsilon floor in future experiments would allow a cleaner separation between policy-directed and randomly-directed coverage, though in the exploration domain random actions are less disruptive than in the hunger domain because an unvisited cell is still rewarded regardless of how the agent arrived at it.

6.4 Survival Instinct Agent

The survival agent was trained for 2,000 episodes in the 8x8 grid with four danger zones and two safe zones. Reward improved by 341% from the first 50 episodes to the final 50 episodes, from an average of 6.07 to 26.80. Danger hit rate fell from around 24% in early training to between 6% and 16% in late training. This improvement reflects the combined effect of danger avoidance learning and energy management: the agent learns both to avoid danger zones and to seek safe zones before its energy is fully depleted.

The Q-table for the survival agent settled at approximately 7,700 unique state keys. This is considerably larger than the hunger and exploration agents, reflecting the additional complexity of the state key, which must encode two danger positions, a safe zone position, and an energy level simultaneously. The energy level is discretised into five bins, which effectively multiplies the state space by five compared to an energy-free state key. The average visits per state at 7,700 states over 2,000 episodes of 60 steps each is approximately 15, which is lower than for

the other agents. This suggests that the survival agent has somewhat less reliable Q-value estimates than the hunger agent, which is consistent with the higher variability observed in testing.

Test performance showed the highest variability of any individual agent. Episode 2 achieved full survival success with 60 steps survived, visiting the safe zone 12 times and maintaining energy throughout. Episode 3 contacted danger on the second step and ended immediately, producing a total reward of negative 4. This extreme outcome, success versus near-immediate failure within the same test session, reflects the sensitivity of the survival task to starting position and early random actions. An agent starting adjacent to a danger zone with a 5% random action probability has a meaningful chance of contacting danger in the first few steps even when its learned policy correctly identifies the danger direction.

6.5 Multi-Instinct Comparative Results

Table 3 shows the performance of the Version 2 multi-instinct agent across the four weight configurations, each trained for 2,000 episodes in the shared 10x10 multi-instinct environment.

Table 3: Version 2 multi-instinct agent results. Ranges reflect variation across separate runs.

Configuration	Avg food/ep	Danger hit rate	Coverage	Survival success
Exploration only	0.0	22 to 28%	18 to 24%	No
Hunger only	0.28 to 0.34	18 to 24%	12 to 16%	No
Survival only	0.0	8 to 16%	8 to 12%	No
All combined	0.05 to 0.12	12 to 18%	14 to 18%	Yes (1+ per 3 test eps)

The most notable result in Table 3 is that only the all-combined configuration ever achieved the survival success condition. Survival success in the multi-instinct environment requires the agent to avoid four danger zones, reach a safe zone at least once, and maintain positive energy for at least 30 consecutive steps. No single-instinct configuration of the multi-instinct agent satisfied this composite criterion across multiple test sessions.

This result is the clearest evidence in favour of the research hypothesis. The composite survival behaviour observed in the all-combined configuration is not a simple superposition of the three individual instinct behaviours; it is a qualitatively different pattern that only emerges when all three signals interact simultaneously.

The survival-only configuration, despite having the danger avoidance and safe zone signals active, did not consistently achieve survival success. Adding the hunger and exploration signals appears to encourage the agent to move more actively through the environment rather than staying in one safe region, which paradoxically improves the agent's ability to find and reach safe zones across diverse starting positions.

The combined configuration also shows a lower average danger hit rate, at 12 to 18%, compared to the exploration-only configuration at 22 to 28% and the hunger-only configuration at 18 to 24%. The survival component is clearly contributing to danger avoidance even in the combined setting, reducing the hit rate below what the exploration or hunger signals alone would produce. However, the combined configuration's danger hit rate is higher than the survival-only configuration's rate of 8 to 16%, suggesting that the hunger signal, which actively directs the agent toward food that may be located near danger zones, does increase the agent's exposure to danger.

The hunger-only configuration in the multi-instinct environment collected an average of 0.28 to 0.34 food items per episode, compared to the hunger specialist agent's 2.92 average in its dedicated environment. This large gap reflects two compounding factors. First, the multi-instinct environment is a 10x10 grid, whereas the hunger specialist operates in an 8x8 grid; the larger grid means the agent must travel further to reach food. Second, the state representation for the multi-instinct environment must encode all objects simultaneously, producing a more complex state space that is harder to cover in 2,000 episodes than the specialist's 392-key space. The drop from 2.92 to 0.31 is therefore not evidence of a failure of the combined architecture; it is a consequence of the harder environment and richer state space.

6.6 Discussion: Trade-Off Between Specialisation and Breadth

The experimental results collectively support a clear conclusion about the trade-off between instinct specialisation and instinct combination. Individual specialist agents, each trained with a single focused reward signal in a purpose-built environment, achieve superior performance on their specific sub-task metric. The hunger specialist reaches 77% food collection success; the combined agent in the hunger-only multi-instinct configuration averages less than one-third of a food item per episode. The exploration specialist covers 33% of the grid on average; the exploration-only multi-instinct configuration covers 18 to 24%. The survival specialist reduces danger hit rates to 6 to 16%; the combined configuration sits at 12 to 18%.

This trade-off has a clear mechanistic explanation. Specialisation works well when a compact state representation can be constructed that is relevant only to the task at hand. The hunger specialist needs to know about food and danger positions relative to the agent; it needs nothing else. This produces a small state space that can be thoroughly covered in a modest episode budget. The multi-instinct agent must encode food, danger, safe zones, and exploration progress simultaneously, producing a far larger state space that requires substantially more training to cover adequately. With the same 2,000-episode budget, the multi-instinct configurations have fewer visits per state and therefore less reliable Q-value estimates.

The question is whether the emergent composite behaviour produced by combination justifies this cost. For the survival success criterion, it clearly does: the specialist cannot achieve it at all. For individual sub-task metrics, it does not. The answer therefore depends on what the application requires. If the goal is maximum food collection in a simple environment, a specialist is the better choice. If the goal is robust behaviour across a complex environment with multiple simultaneous demands, combination may be necessary even at the cost of specialised performance.

The finding that the survival-only multi-instinct configuration fails to achieve survival success while the all-combined configuration succeeds is particularly noteworthy. It suggests that the food collection and exploration components contribute to the agent's ability to meet the survival criterion in indirect ways. The food signal may increase the frequency of active navigation, reducing the probability that the agent remains stationary in an energy-draining state. The exploration signal may increase the agent's willingness to move toward unfamiliar regions, which include safe zones it has not yet visited. These are speculative explanations; confirming them would require ablation studies that systematically remove individual components while holding others constant.

6.7 Limitations

Several factors limit the conclusions that can be drawn from this evaluation. The most important is that the multi-instinct environment is more difficult than any individual specialist environment, so comparisons of absolute metric values between the specialists and the multi-instinct configurations conflate the effect of instinct combination with the effect of environment difficulty. A fully controlled comparison would evaluate the specialist agents within the multi-instinct environment, keeping environment difficulty constant while varying whether single or multiple instincts are active. This comparison is identified as the first priority for future work.

A second limitation is the small test sample. Three test episodes per agent is insufficient to produce stable estimates of success rates and average performance metrics. The 95% confidence interval on a binomial proportion estimated from three trials is extremely wide; a single episode difference corresponds to a 33 percentage point change in the estimated success rate. The wide confidence intervals mean that the differences observed between configurations may not be statistically significant in a formal sense. Increasing the test sample to 30 or 100 episodes would dramatically improve the reliability of the estimates without requiring additional training.

A third limitation is the use of equal instinct weights in the combined configuration. The three reward signals have different magnitudes and different frequencies of occurrence, so a weight of 1.0 for each does not treat them as genuinely equal in their influence on the learned policy. A food collection event contributes 10.0 to the combined reward, while a survival step contributes 1.0. Over 60 steps, the survival signal contributes approximately 60.0 in a perfect run, while food collection contributes 30.0 for three collections. The relative magnitudes suggest that the survival signal may dominate the combined reward in most episodes, and the apparent equal weighting is not genuinely equal in effect. A systematic grid search over weight configurations would provide a more informative picture of the trade-off surface between the three instincts.

A fourth limitation concerns the generalisability of the results to environments of different sizes. All experiments are conducted on fixed grid sizes (8x8 or 10x10). The relative encoding scheme is designed to be scale-independent in principle, because it encodes direction and distance bin rather than absolute coordinates. However, the distance bins were calibrated for these specific grid sizes, and a grid substantially larger or smaller might require different bin thresholds. Testing the trained policies on grids of different sizes, or training on grids of varying sizes and evaluating on a held-out size, would address the robustness objective identified in the interim report.

Finally, the project uses tabular Q-learning throughout. This method requires explicit state visits for convergence and does not generalise across states. Deep reinforcement learning methods, particularly DQN (Mnih et al., 2015) or actor-critic approaches, would learn a function approximation that can produce reasonable Q-values for states not explicitly visited during training, by generalising from nearby states in the representation space. This would substantially increase the scalability of the approach, allowing larger environments, more objects, and more complex instinct interactions to be studied. The tabular limitation is therefore not merely a

computational inconvenience but a fundamental constraint on the complexity of experiments that can be conducted within the current framework.

7 External Concerns

7.1 Legal, Social and Ethical Issues

This project is a simulation study conducted entirely within a controlled computational environment. No human participants were involved at any stage, no personal data of any kind was collected or processed, and no system was deployed in a live or production setting. The reinforcement learning agents exist solely as software processes that interact with simulated environments implemented as Python objects. There is accordingly no requirement for ethical review beyond the standard academic research conduct obligations.

All code in the project is original work by the author. The third-party libraries used, specifically NumPy for numerical computation, OpenAI Gymnasium for environment management, and matplotlib for visualisation, are open-source software distributed under permissive licences that allow use in academic projects without restriction. All external academic sources cited in this report have been consulted through the University of Hull library system or via publicly accessible preprint repositories, consistent with the University's policies on academic integrity and copyright.

The broader concept of instinct-driven artificial agents raises questions that are worth acknowledging even at the research stage. Survival and avoidance instincts modelled after biological drives could, in principle, be incorporated into autonomous systems operating in real environments. A robot equipped with a hunger-equivalent resource-seeking drive and a self-preservation drive would have persistent motivations that operate independently of explicit task instructions, and that could produce behaviours in novel situations that the designer did not anticipate. This is qualitatively different from a task-conditioned system that acts only in response to explicit commands.

The project also touches on the question of whether biologically inspired AI architectures might produce systems that are harder to interpret or control than systems trained with a single explicit objective. When multiple instinct signals interact, the resulting behaviour may be difficult to predict from the individual instinct policies alone, as demonstrated by the emergent survival success behaviour in the combined agent. In a deployed system, such emergent behaviour would need to be characterised carefully before deployment to ensure that the

combined policy does not produce unintended outcomes in edge cases. This concern is not specific to the architecture used here but applies generally to systems where behaviour emerges from the interaction of multiple independent components rather than from a single unified design.

7.2 Commercial Considerations

The modular architecture developed in this project has potential commercial value in several application areas. The separation of environment logic, reward functions, and agent classes into distinct components means that new instincts can be added by implementing a new reward function and registering it in the combination mechanism, without modifying any existing code. This extensibility would be attractive in commercial settings where new requirements emerge iteratively and the cost of modifying a monolithic codebase is high.

In game development, non-player characters driven by competing instinct signals could exhibit more believable and varied behaviour than characters scripted with explicit rule sets. A character with hunger, curiosity, and self-preservation drives would navigate the game world in ways that feel organic and context-sensitive, reacting differently to the same environmental configuration depending on its current drive states. The unpredictability introduced by drive competition would be a feature rather than a bug in this context, providing players with more engaging adversarial or companion behaviour.

In robotics research, a similar architecture could provide a framework for combining resource-seeking, obstacle-avoidance, and exploration behaviours without requiring a single monolithic reward function. The key research challenge would be scaling the combination mechanism beyond linear scalarisation to handle the temporal dependencies inherent in physical robot navigation, where the consequences of instinct conflicts unfold over many steps rather than resolving within a single grid step.

8 Conclusion

8.1 Project Management

The project followed an iterative development approach, beginning with the exploration instinct as the foundational component and progressively adding the hunger and survival instincts before combining them. This ordering was motivated by the interim report's rationale that exploration is a prerequisite for effective learning in any instinct domain: an agent that does not explore cannot learn about the consequences of its actions in unfamiliar parts of the state space. Developing

the exploration instinct first ensured that the state abstraction and Q-learning infrastructure was in place before the more complex hunger and survival reward functions were added.

Regular meetings with the project supervisor informed key decisions throughout the project. The decision to pivot from Unity ML-Agents to OpenAI Gymnasium was taken before the midpoint of the project, preserving the majority of the implementation schedule for the core research work. The midway review identified the state space problem as the primary technical risk and prompted the development of the relative encoding scheme that resolved it. Without the supervisor input at this stage, the project might have continued with the absolute encoding approach and produced results too noisy to support the central research question.

The Gantt chart in the interim report showed documentation and final report writing planned for March to April 2026. In practice, the implementation phase extended beyond the planned end date, primarily because of the time spent diagnosing the hunger agent's broken reward function and the state encoding problem. The report writing phase was compressed to fit within the remaining time. This schedule overrun is typical of software research projects where debugging and design refinement take longer than anticipated; the interim report's schedule was an optimistic estimate based on the assumption that the initial design would work as intended.

Secondary objectives from the interim report were not completed within the project timeline. These included robustness testing on different grid sizes and exploration of sequential instinct activation strategies. Both remain identified directions for future work. The core objectives, implementing and evaluating all three instincts individually and in combination, were completed and produced results sufficient to address the research question. The primary objectives were therefore achieved within the project timeline despite the schedule overrun on implementation.

8.2 Risk Management

The risk register compiled at the outset of the project identified platform incompatibility as the principal risk, given that Unity ML-Agents had known issues with Apple Silicon hardware at the time. This risk materialised precisely as anticipated. The mitigation strategy of pivoting to OpenAI Gymnasium was executed as planned, and no project time was lost beyond what had been budgeted for the contingency. The principal lesson from this risk management experience is that having a clearly specified and technically evaluated fallback platform before

beginning implementation substantially reduces the cost of a platform failure when it occurs.

A risk that was not explicitly identified in the initial register was the possibility that the Q-learning state representation would produce state spaces too large for convergence within the available episode budget. This problem was encountered during the implementation of the hunger agent and later confirmed for the multi-instinct agent. The state space problem is a known challenge in tabular reinforcement learning, and its omission from the risk register reflects an underestimation of its severity in the specific context of this project's environments. Future projects using tabular methods should include state space tractability as an explicit risk item, with a pre-implementation analysis of the expected state space size under the proposed encoding.

The debugging process for the hunger agent's reward function was more time-consuming than expected. The symptom, an agent that appeared to be learning during training but performed no better than random in testing, was initially attributed to insufficient training episodes. It was only after the training time had been substantially increased without improvement that the reward function was identified as the source of the problem. This diagnostic process would have been faster if a formal testing protocol had been established earlier: a protocol that required Q-value differentiation as a necessary precondition for accepting a training result would have flagged the problem sooner by detecting that the Q-values were not differentiating between actions despite training progress.

8.3 General Conclusions

This project has demonstrated that instinct layering in tabular reinforcement learning produces qualitatively richer composite behaviour than any single instinct configuration tested. The combined multi-instinct agent was the only configuration capable of simultaneously managing food collection, danger avoidance, and energy conservation to the degree required by the survival success criterion. This result directly addresses the research question and supports the hypothesis that instinct combination enables behaviours not achievable by any individual instinct in isolation.

At the same time, individual specialist agents outperformed the combined agent on every specific sub-task metric. The hunger specialist reached 77% success in its dedicated environment; the multi-instinct hunger-only configuration averaged less than half a food item per episode. This reflects a genuine trade-off: the combined architecture sacrifices specialised performance for behavioural breadth. The trade-

off is not a failure of the combined architecture but an inherent consequence of the larger state space and more complex reward landscape that multiple simultaneous instincts produce.

The research question asked whether instinct layering leads to improved learning performance and more realistic behavioural patterns. The answer is that it depends on how improvement is measured. On individual sub-task metrics, the combined agent does not outperform the specialists. On the composite survival criterion, it is the only configuration that can satisfy the criterion at all. On the dimension of behavioural realism, the combined agent exhibits context-sensitive prioritisation of instincts across different environment states, which qualitatively resembles the drive-competition described in biological motivation theory. Whether this constitutes genuinely more realistic behaviour or merely more complex behaviour is a philosophical question beyond the scope of this project.

The four main contributions of this project are: a replicable implementation of an instinct-driven reinforcement learning architecture in OpenAI Gymnasium; an empirical comparison of individual and combined instinct configurations using standardised metrics across consistent training conditions; a practical demonstration that the relative directional state encoding scheme reduces state space size by more than an order of magnitude compared to absolute encoding, enabling convergence of tabular Q-learning within modest episode budgets; and an identification of the survival success criterion as a composite metric that distinguishes instinct combination from single-instinct specialisation. These contributions are modest in scope but concrete and reproducible, providing a foundation for future work on more complex instinct architectures and larger environments.

8.4 Further Work

The following directions are identified as priorities for extending this work.

- Evaluate individual specialist agents within the multi-instinct environment, holding environment difficulty constant, to produce a fair comparison of absolute performance metrics across instinct configurations. This is the most important methodological improvement and would allow the trade-off between specialisation and combination to be cleanly separated from the confounding effect of environment difficulty.
- Apply deep Q-networks (Mnih et al., 2015) or proximal policy optimisation to overcome the state space limitations of tabular Q-learning. Neural function approximation would allow the project to scale to larger, more complex

environments and produce more reliable estimates from fewer visits per state, addressing the primary scalability limitation of the current work.

- Introduce a higher-level arbitrator or meta-policy that learns to weight instinct signals dynamically based on the current environment state, rather than using fixed linear scalarisation. This would address the most significant architectural limitation of the current combined agent and move the design closer to the biological model of context-sensitive drive prioritisation.
- Conduct a systematic search over instinct weight configurations to identify better balanced combinations and understand how reward magnitude differences between instincts affect which drive dominates the combined policy. The equal-weight configuration used in this project is unlikely to be optimal and may substantially underestimate the performance achievable by tuned combination.
- Test whether policies learned on an 8x8 or 10x10 grid transfer to grids of different sizes, addressing the robustness objective identified in the interim report as a secondary aim. The relative encoding scheme is designed to be scale-independent in principle, and this should be verified empirically.
- Explore sequential instinct activation, where instincts are introduced progressively during training rather than simultaneously, to determine whether a curriculum over instinct complexity produces better convergence in the combined environment. Curriculum learning has been shown to improve learning stability in other reinforcement learning contexts and may address the challenge of the combined agent's larger state space requiring more training to cover adequately.

References

- Albrecht, S.V., Christianos, F. and Schäfer, L. (2024). Multi-Agent Reinforcement Learning: Foundations and Modern Approaches. Cambridge, MA: MIT Press.
- Andres, A. and Del Ser, J. (2024). Fostering intrinsic motivation in reinforcement learning with pretrained foundation models. arXiv preprint arXiv:2410.07404.
- Berlyne, D.E. (1960). Conflict, Arousal, and Curiosity. New York: McGraw-Hill.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J. and Zaremba, W. (2016). OpenAI Gym. Available at: <https://arxiv.org/abs/1606.01540> (Accessed: 13 May 2026).
- Chadi, M. and Mousannif, H. (2023). Understanding reinforcement learning algorithms: the progress from basic Q-learning to proximal policy optimisation. arXiv preprint arXiv:2304.00026.
- Ghasemi, M. et al. (2025). A comprehensive survey of reinforcement learning: from algorithms to practical challenges. arXiv preprint arXiv:2411.18892.
- Huh, D. et al. (2024). Multi-agent reinforcement learning: a comprehensive survey. arXiv preprint arXiv:2312.10256.
- Hull, C.L. (1943). Principles of Behavior: An Introduction to Behavior Theory. New York: Appleton-Century-Crofts.
- Li, L., Walsh, T.J. and Littman, M.L. (2006). Towards a unified theory of state abstraction for MDPs. Proceedings of the International Symposium on Artificial Intelligence and Mathematics, pp. 531-539.
- Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P. and Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. Advances in Neural Information Processing Systems, 30.
- Milvus (n.d.). What is intrinsic motivation in reinforcement learning? Available at: <https://milvus.io> (Accessed: 13 May 2026).
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G. and Petersen, S. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), pp. 529-533.
- Ng, A.Y., Harada, D. and Russell, S. (1999). Policy invariance under reward transformations: theory and application to reward shaping. Proceedings of the Sixteenth International Conference on Machine Learning, pp. 278-287.

- Oudeyer, P.-Y. and Kaplan, F. (2009). What is intrinsic motivation? A typology of computational approaches. *Frontiers in Neurorobotics*, 3.
- Pathak, D., Agrawal, P., Efros, A.A. and Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. *Proceedings of the 34th International Conference on Machine Learning*, pp. 2778-2787.
- Roijers, D.M., Vamplew, P., Whiteson, S. and Dazeley, R. (2013). A survey of multi-objective sequential decision-making. *Journal of Artificial Intelligence Research*, 48, pp. 67-113.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Stephens, D.W. and Krebs, J.R. (1986). *Foraging Theory*. Princeton, NJ: Princeton University Press.
- Sutton, R.S. and Barto, A.G. (2018). *Reinforcement Learning: An Introduction*. 2nd edn. Cambridge, MA: MIT Press.
- Sutton, R.S., Precup, D. and Singh, S. (1999). Between MDPs and semi-MDPs: a framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1-2), pp. 181-211.
- Watkins, C.J.C.H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3-4), pp. 279-292.
- Yu, L. (n.d.). MARL-Papers: a paper list of multi-agent reinforcement learning. Available at: <https://github.com/LantaoYu/MARL-Papers> (Accessed: 13 May 2026).